

EVERYTHING EVERYWHERE ALL AT ONCE: LLMs CAN IN-CONTEXT LEARN MULTIPLE TASKS IN SUPERPOSITION

Zheyang Xiong^w, Ziyang Cai^w, John Cooper^w, Albert Ge^w, Vasilis Papageorgiou^w
Zack Sifakis^w, Angeliki Giannou^w, Ziqian Lin^w, Liu Yang^w, Saurabh Agarwal^w
Grigorios G Chrysos^w, Samet Oymak^m, Kangwook Lee^w, Dimitris Papailiopoulos^{w,ms}

^wUniversity of Wisconsin-Madison, ^mUniversity of Michigan, ^{ms}Microsoft Research

ABSTRACT

Large Language Models (LLMs) have demonstrated remarkable in-context learning (ICL) capabilities. In this study, we explore a surprising phenomenon related to ICL: LLMs can perform multiple, computationally distinct ICL tasks simultaneously, during a single inference call, a capability we term “task superposition”. We provide empirical evidence of this phenomenon across various LLM families and scales and show that this phenomenon emerges even if we train the model to in-context learn one task at a time. We offer theoretical explanations that this capability is well within the expressive power of transformers. We also explore how LLMs internally compose task vectors during superposition. Furthermore, we show that larger models can solve more ICL tasks in parallel, and better calibrate their output distribution. Our findings offer insights into the latent capabilities of LLMs, further substantiate the perspective of “LLMs as superposition of simulators”, and raise questions about the mechanisms enabling simultaneous task execution.

1 INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities across various domains, with one of the most intriguing being in-context learning (ICL). ICL enables LLMs to perform tasks during inference without the need to fine-tune for that particular task, simply by providing a few examples within the input prompt. This ability has sparked significant interest in the research community, as it suggests that LLMs can adapt to novel tasks on-the-fly, using the capabilities that they acquired during pretraining, and the context provided.

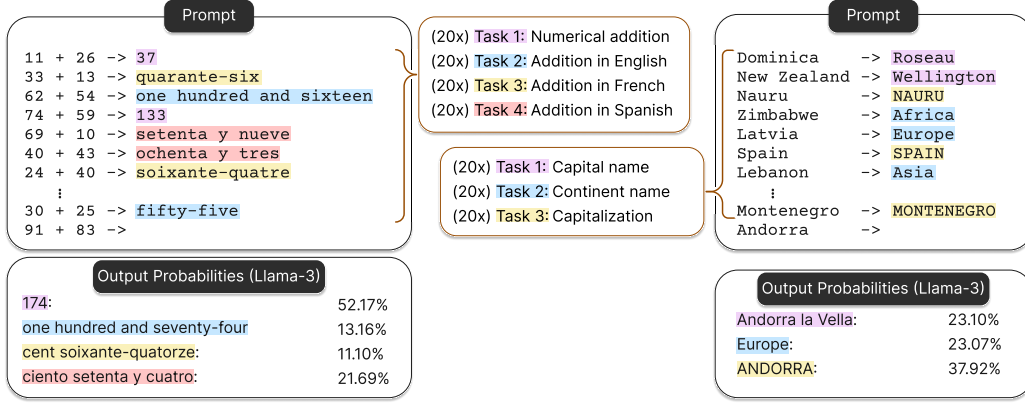
While ICL has been extensively studied from both theoretical and empirical perspectives, many aspects of its underlying mechanisms remain elusive. In this work, we study a surprising phenomenon related to ICL that, to the best of our knowledge, has not been thoroughly studied before: LLMs can perform multiple distinct ICL tasks simultaneously, in a single inference call, a capability we refer to as “*task superposition*”, with examples shown in Figure 1.

Our study suggests that pretrained autoregressive LLMs such as Llama (Touvron et al., 2023) or GPT-3.5¹ (Brown et al., 2020) display superposition of tasks purely *in-context*. When presented with multiple in-context examples from different tasks, in the same prompt, the models can generate outputs that correspond to solutions for all these individual tasks. For instance, given examples of addition and translation, the model can concurrently produce correct answers for both tasks, as well as the composition of these tasks (e.g., the result of addition translated into another language).

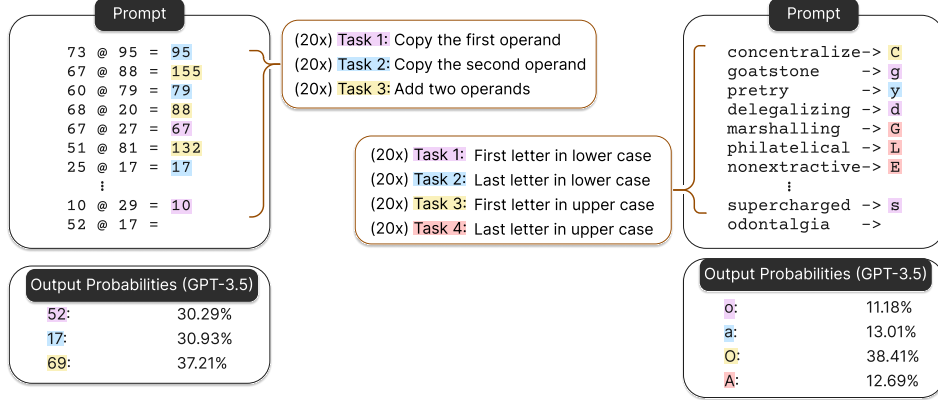
Figure 1 illustrates this phenomenon. In Figure 1a (left), given in-context examples of addition in different languages and the query “91 + 83 →”, the model generates probabilities for the correct sum in various languages, demonstrating its ability to perform addition and translation concurrently.

Email: <zheyang@cs.wisc.edu>. Correspondence: <dimitris@papail.io>.
Code available at: github.com/edixiong/task-superposition

¹In particular, gpt-3.5-turbo-instruct.



(a) **(left)** Two-digit addition in a variety of languages. **(right)** Naming the capital of a given country name, naming the continent of a given country name or capitalizing the country name.



(b) **(left)** Tasks copy (op1), copy (op2) and op1+op2. **(right)** First or last letter in upper or lower case.

Figure 1: LLMs can perform task superposition. **(a)** Llama-3 70B and **(b)** GPT-3.5 Turbo are each presented with two sets of tasks. For each set of tasks, we show an example prompt such that all except the last row are in-context examples of one of the tasks and the last row is the query. We provide 20 in-context task examples for each task in the prompt with order randomized and provide the probabilities of outputs when correctly performing each task on the query.

This discovery aligns and lends further support to the view of LLMs as superposition of simulators (Janus, 2022; Shanahan et al., 2023; Nardo, 2023) and the Bayesian perspective of ICL proposed by Xie et al. (2022). While not a mathematically rigorous formulation, we can conceptualize the output of an LLM as a weighted sum of conditional probabilities across possible tasks:

$$\mathbb{P}(\text{output}|\text{prompt}) \approx \sum_{\text{task}} \mathbb{P}(\text{output}|\text{task}, \text{prompt}) \mathbb{P}(\text{task}|\text{prompt}).$$

In this conceptual model, $\mathbb{P}(\text{output}|\text{prompt})$ represents the probability distribution over possible outputs given the input prompt, a task can be thought of as a latent variable representing different capabilities the model might possess (e.g., arithmetic, translation, sentiment analysis), $\mathbb{P}(\text{output}|\text{task}, \text{prompt})$ represents the output probability distribution if the model was specifically attempting to solve a single task, based on the test example in the prompt, and $\mathbb{P}(\text{task}|\text{prompt})$ represents the model’s inferred probability that the prompt specifies a particular task.

Although this mental model is an over-simplification of how an LLM operates, it offers a clean conceptual framework for the task superposition phenomenon we observe. Our findings lend support to the idea that LLMs can simultaneously maintain and utilize multiple task distributions, resulting in outputs that reflect a combination of relevant tasks.

Our Contributions: Our study makes several key contributions:

1. Through extensive empirical investigation and theoretical results, we demonstrate that task superposition is prevalent across various pretrained LLM families (GPT3.5, LLama-3, Qwen).
2. We empirically show that task superposition can emerge, even if we train on one task at a time.
3. We provide a theoretical construction showing that Transformers models are indeed capable of task superposition, and have the capacity to implement multiple tasks in parallel.
4. We explore how LLMs internally compose task vectors (Hendel et al., 2023) during superposition, and show how convex combinations of task vectors can reproduce the superposition effect.
5. We show that larger models can solve more tasks in parallel and more accurately reflect the distribution of in-context tasks.

We believe that our findings offer new insights into the latent capabilities of LLMs and raise questions about the mechanisms enabling simultaneous task execution. We believe this work sheds more light on the ICL capabilities of frontier language models, and offers a glimpse on potential applications of task superposition in practical settings.

2 RELATED WORK

Theory and practice of in-context learning. There is rich literature which formalizes in-context learning under diverse definitions. For example, prior works study in-context learning through a Bayesian framework for task retrieval (Xie et al., 2022; Panwar et al., 2023; Zhang et al., 2023), martingales (Falck et al., 2024), optimizers (Akyürek et al., 2023; Oswald et al., 2023; Dai et al., 2022) and more (Reddy, 2024; Olsson et al., 2022). Other works confirm the theoretical framing of in-context learning by using it to implement a variety of algorithms and methods (Zhou et al., 2023; Ahn et al., 2023; Giannou et al., 2023; Wu et al., 2024; Laskin et al., 2022; Zhou et al., 2022), or to approximate general-purpose computing machines (Giannou et al., 2023; Wei et al., 2022).

To bridge the gap between theory and practice, many works have used these theoretical insights to study in-context learning behaviors, such as in many-shot in-context learning, (Agarwal et al., 2024), long-context (Li et al., 2024), or eliciting personas (Choi & Li, 2024). Other works study the factors that influence how well models can learn through context, such as task diversity (Raventos et al., 2023; Chan et al., 2022), the balance between pre-training priors and in-context (Wei et al., 2023; Lin & Lee, 2024), in-context labels (Min et al., 2022; Lyu et al., 2022), and the in-context format (Lampinen et al., 2022). In-context learning has also been proposed as a means of fine-tuning to improve non-language tasks (Dinh et al., 2022).

The development of new architectures such as state space models (Gu & Dao, 2023) has further motivated studying whether in-context learning is prevalent in alternative architectures such as Mamba (Park et al., 2024; Grazzi et al., 2024; Zeng et al., 2024) or in looped transformers (Yang et al., 2023).

Steering models through in-context learning has been a growing area of interest. Recent work has hypothesized that in-context learning can be encapsulated by a high-dimensional description of a task, which can be used to replace, (Hendel et al., 2023) compose (Todd et al., 2024) or augment (Liu et al., 2024) the latent states of a model, in order to alter its default behavior. Task vectors can be combined via arithmetic operations to solve a variety of tasks (Ilharco et al., 2023). Prior work has also been investigating the power of tokens in defining a task (Bai et al., 2024).

Other definitions of superposition. Our findings on superposition are inspired by notions of language models as multiverse generators (Reynolds & McDonell, 2021; moire, 2021). One consequence of LLMs as a superposition of tasks is that the outputs may collapse to unintended simulacra, a behavior known as the “Waluigi effect” (Nardo, 2023).

Superposition has been defined in various related contexts of learning models. Feature superposition (Elhage et al., 2022) refers to a neural network’s ability to represent multiple learned concept in a single neuron. Though our discovery of task superposition describes the same abstract idea, we stress that it is distinct from feature superposition because task superposition is most apparent in the final output of a model. Feature superposition is a microscopic-level observation whereas task superposition is a macroscopic-level observation.

Superposition is also described as a way to store multiple models in a single set of parameters (Cheung et al., 2019), processing multiple inputs simultaneously (Shen et al., 2024a; Murahari et al.,

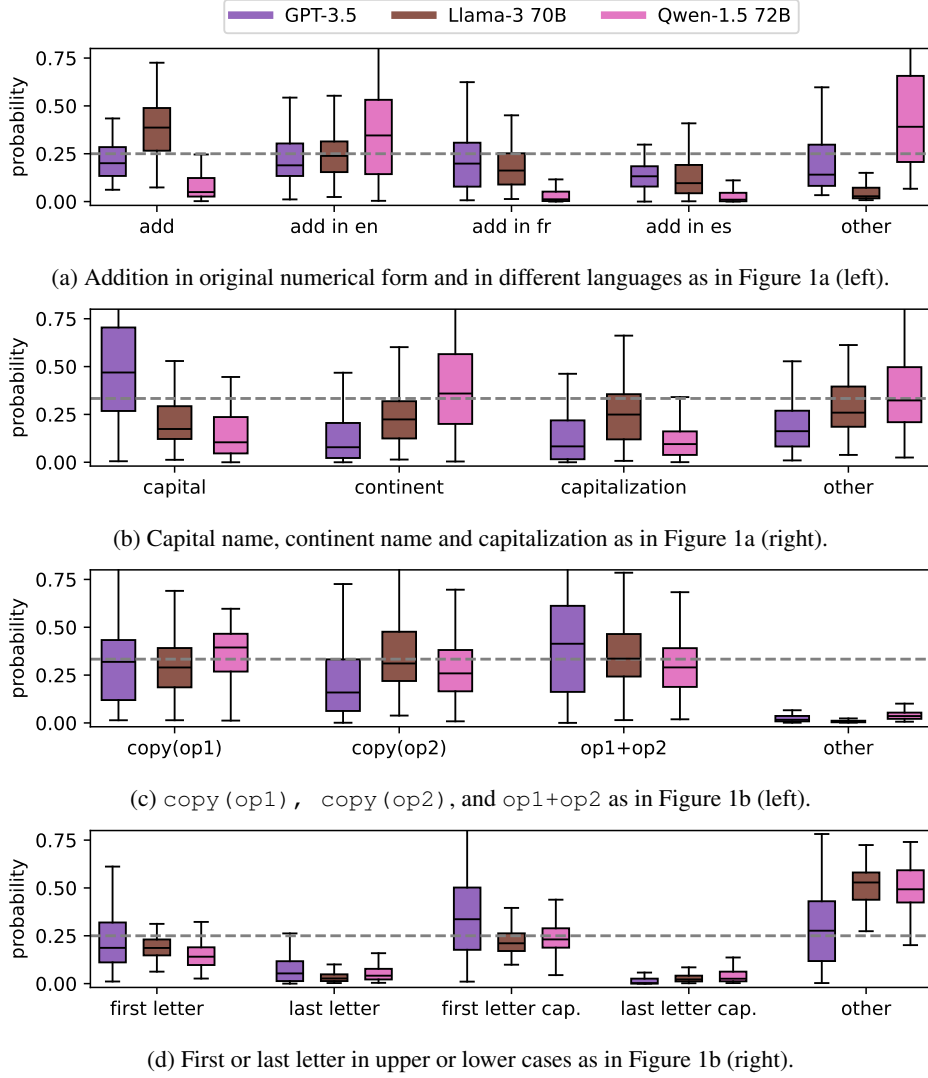


Figure 2: Distributions of probabilities for correct outputs for each task in box plots, where 0/25/50/75/100-percentiles are shown and region between 25 and 75 percentiles is colored. For every set of tasks, we tested with 100 prompts and for each prompt, every task has 20 random in-context task examples with order randomized like in Figure 1. Category *other* is the sum of probabilities of all other outputs. Gray dashed line in each figure is the ideal probability if we assume the model perfectly calibrates its output distribution to the distribution of in-context task examples. With uniform distribution of task examples, the dashed lines are at 0.25 (4 tasks setting) and 0.33 (3 tasks setting).

2022). In our work, we demonstrate task superposition directly as a result of language pre-training, without the necessity of additional adapters or decoding strategies.

3 LLMs ARE A SUPERPOSITION OF MULTIPLE IN-CONTEXT LEARNERS

In this section, we want to investigate if existing pre-trained models exhibit superposition of multiple tasks and whether this phenomenon is common (i.e., whether we can observe this phenomenon on a variety set of tasks and different families of LLMs).

Finding 1: *LLMs can in-context learn multiple tasks in superposition when provided with prompts of a mixture of task examples.*

We denote K by the number of tasks and consider four different settings of task mixtures.

1. Numerical addition and addition in English, French or Spanish ($K = 4$). Example prompt is shown in Figure 1a (left).
2. Given a name of a country, name the capital, continent or capitalize the country name ($K = 3$). Example prompt is shown in Figure 1a (right).
3. Given input “{op1}@{op2}”, copy op1, op2 or add op1 and op2 ($K = 3$). Example prompt is shown in Figure 1b (left).
4. Given a word, output first letter or last letter in lower or upper cases ($K = 4$). Example prompt is shown in Figure 1b (right).

We provide GPT-3.5 (Brown et al., 2020), Llama-3 70B (AI@Meta, 2024) and Qwen-1.5 72B (Bai et al., 2023a) with prompts of uniform mixture of tasks (each task has 20 random examples in the prompt ordered randomly). For each prompt consisting of in-context task examples (e.g., “11 + 26 → 37” for the first task in the first setting) and a query (e.g., “91 + 83 →”), we calculate the probabilities of outputs when correctly performing each task on the query and plot the distribution of probabilities for each task in Figure 2. Details on calculating the probabilities is in Appendix B.

Figure 2 reveals that in all four sets of tasks, all models have non-negligible median values of probabilities for at least two tasks. This indicates that the models can in-context learn multiple tasks in superposition when provided with prompts of a mixture of task examples.

We can also observe that, even though every task in a prompt has an equal number of in-context examples (20 examples), LLMs do not calibrate their output distribution perfectly with the in-context task example distribution and they still have bias on what task to perform. For example, Figure 2a shows that Llama-3 70B prefers performing numerical addition over addition in other languages, Qwen-1.5 72B prefers addition in English while GPT-3.5 does not have a strong preference over a single task. On the other hand, in Figure 2b GPT-3.5 has a strong preference over the capital task.

Additionally, some tasks are “harder” than other tasks. For example, in Figure 2d, all models assign near-zero probability for task answers of `last_letter` and `last_letter_cap`. The category `other` has relatively high values, indicating a high noise when prompted with in-context examples of this setting. In contrast, in Figure 2c, category `other` has very small values, indicating that all models most of the time would correctly assign the output probabilities to the correct answers.

4 TASK SUPERPOSITION IN MODELS TRAINED FROM SCRATCH

In Section 3 we investigated task superposition in pre-trained LLMs at inference time. In this section, we further investigate how task superposition emerges in LLMs during training. Specifically, if we train the model to in-context learn one task at a time, can it perform task superposition when provided with prompts containing examples of multiple tasks?

To answer this question, we train a small GPT-2 model (12 heads, 12 layers and ~86 million parameters) (Radford et al., 2019) to learn a family of retrieval tasks. The input has the form “{ch1}{ch2}{ch3}{ch4}{ch5}{ch6}{ch7}{ch8}→” where `ch1`, ..., `ch8` are distinct single characters. We consider 8 retrieval tasks – `ret1`, ..., `ret8` – where `ret1` is to output `ch1` and so on. The model is trained to in-context learn one task (retrieve one of {`ch1`, ..., `ch8`}) at a time in training. Namely, during training, the model is only provided with text data such that each prompt only contains in-context examples of a single randomly chosen task (and different prompts can correspond to different tasks).

Concretely, for each sample, we randomly select task $t \in \{\text{ret1}, \dots, \text{ret8}\}$ and inputs $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$, where each $\mathbf{x}^{(j)}$ is an eight-character long string. We then form the sequence $\mathbf{s} = [\mathbf{x}^{(1)}, \mathbf{g}_t(\mathbf{x}^{(1)}), \dots, \mathbf{x}^{(m)}, \mathbf{g}_t(\mathbf{x}^{(m)})]$ where $\mathbf{g}_t(\mathbf{x}^{(j)})$ is the output of performing task t on $\mathbf{x}^{(j)}$. We train the model M_θ parametrized by θ using ICL training. In particular, we minimize the following objective:

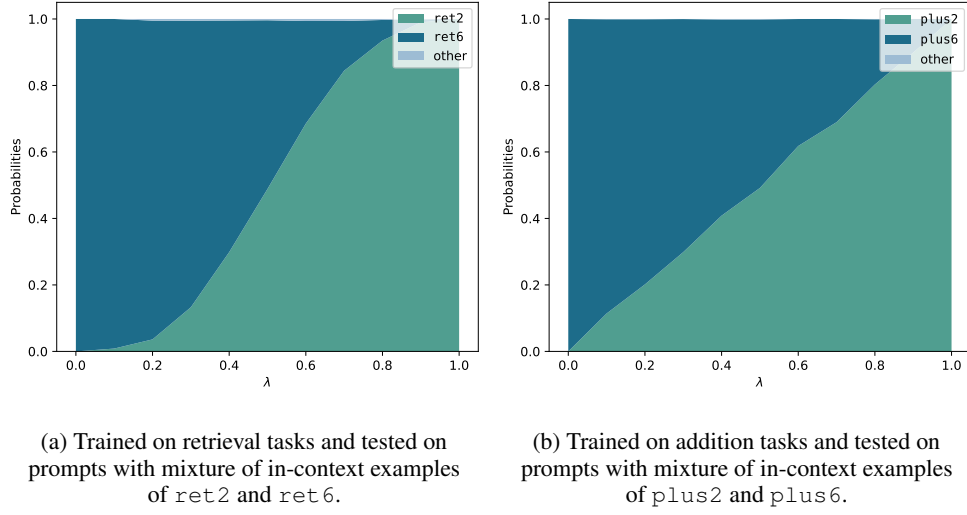


Figure 3: We consider two different training settings of tasks: (a) given an eight-character length string as input, consider `ret1`, ..., `ret8` where `ret1` is to retrieve the first character and so on; and (b) given a two-digit integer as an input, consider `plus0`, ..., `plus9` where `plus0` is to add 0 on the input and so on. The model in-context learn one task at a time during training. After training, for each setting, we select two tasks and we provide the model with prompts containing in-context examples from these two tasks and vary the mixture ratio λ such that the in-context task example distribution for two tasks is $[\lambda, 1 - \lambda]$. We plot λ on x-axis and the output probabilities of task answers for each task on y-axis.

$$\min_{\theta} \mathbf{E}_s \left(\frac{1}{m-1} \sum_{j=1}^{m-1} \text{CE}(M_{\theta}(s_j \oplus \mathbf{x}^{(j+1)}), g_t(\mathbf{x}^{(j+1)})) \right), \quad (1)$$

where $s_j \oplus \mathbf{x}^{(j+1)} \equiv [\mathbf{x}^{(1)}, g_t(\mathbf{x}^{(1)}), \dots, \mathbf{x}^{(j)}, g_t(\mathbf{x}^{(j)}), \mathbf{x}^{(j+1)}]$ and CE is the cross-entropy loss.

After training, we provide the model with prompts containing in-context examples of two tasks (in particular, we choose `ret2` and `ret6`) and see if the model performs task superposition. We vary the proportion of in-context examples of two tasks and plot the output distributions in Figure 3a.

Similarly, we consider a second setting involving 10 tasks. Given a two digit integer input `num`, task `plus0` outputs `num`, task `plus1` outputs `num + 1` and so on, up to task `plus9`. The model is trained to in-context learn one of `plus0`, ..., `plus9` at a time, following the procedure above. During inference time, the model is tested with prompts containing a mixture of in-context examples from tasks `plus2` and `plus6`. We vary the mixture ratio and show the output distributions in Figure 3b.

Finding 2: *Transformers can in-context learn multiple tasks in superposition even if trained to in-context learn one task at a time.*

Remarkably, from Figure 3a and 3b, GPT-2 trained from scratch that in-context learns one task at a time can generalize to simultaneously performing multiple tasks and calibrate the output probabilities according to the in-context task example distribution when provided with a mixture of in-context examples. For example, in Figure 3a at the mixture ratio $\lambda = 0.5$, meaning that 50 percent of the examples in the prompt is from task `ret2` and the other 50 percent comes from task `ret6`, we can see the output probabilities for task answers of `ret2` and `ret6` being roughly $[0.5, 0.5]$. We can also observe similar behavior in Figure 3b.

5 TRANSFORMERS HAVE THE CAPACITY TO PERFORM TASK SUPERPOSITION

In this section, we explore whether Transformers have the inherent expressivity to perform multiple tasks in superposition with a single inference call. To this end, we provide a theoretical construction

of a Transformer which, given the ability to implement multiple tasks, performs task superposition depending on the examples given in-context.

Theorem 1. *A seven layer transformer with embedding dimension $\mathcal{O}(d + \log(mn))$ with K heads per attention layer can perform K tasks on vectors of dimension d in superposition, with weighting based on m different in-context examples each of length n .*

The proof of Theorem 1 is provided in Appendix D.4. Note that while this does not guarantee that training a Transformer will actually find these parameters, it does indicate that Transformers are expressive enough to perform task superposition at test time. Below we outline the main ideas used in the proof.

Prediction based on multiple tasks. Assume that we are given m in-context samples $(\mathbf{x}_1^{(j)}, \dots, \mathbf{x}_{n-2}^{(j)}, '=', \mathbf{y}^{(j)})_{j=1}^m$ where '=' represents a specific value used only for preceding the label, and a set of k different Transformers TF_i which can implement the T different desired tasks, where each deterministic task is denoted as $\mathbf{g}_i(\mathbf{x}^{(j)})$ with $i \in [k]$ and $j \in [m]$, i.e. $\mathbf{y}^{(j)} = \mathbf{g}_i(\mathbf{x}^{(j)})$ for some task i dependent on sample j . Using the weights of each TF_i , we can compute outputs of the following form:

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \end{bmatrix} \rightarrow \begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \|\mathbf{g}_1(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \|\mathbf{g}_T(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \end{bmatrix}$$

We use the l_1 norm to aggregate the prediction, in case that the task is multi-dimensional. These differences are used to identify tasks, as $\|\mathbf{g}_i(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 \approx 0$ for $\mathbf{y}^{(j)}$ coming from task i . Different heads at each layer in the model are used to execute each of the tasks in parallel using the weights from TF_i . In Appendix D we construct tasks where an arbitrary function $\mathbf{g}_i(\mathbf{x}_l^{(j)})$ is implemented using ReLUs for some fixed l that is task-specific.

Creating task identifiers. Having the differences between the implemented function and the label, we first use the ReLUs to clean up the vectors $\mathbf{v}_k$ so that only the positions in each vector that are associated with a task are maintained and the rest are set to 1². We thus create the vectors $(\mathbf{v}'_k)_i = \|\mathbf{g}_k(\mathbf{x}_{1:l-1}^{(j)}) - \mathbf{x}_l^{(j)}\|_1$ and $(\mathbf{v}'_k)_* = 1$ otherwise. Now we use ReLUs to threshold and create an indicator vectors $\mathbf{1}_{\{\|\mathbf{g}_k(\mathbf{x}_{1:l-1}^{(j)}) - \mathbf{x}_l^{(j)}\|_1 \approx 0\}}$ which identify the task, i.e., these are task identifiers.

Notice that if the task is correctly predicted then the difference should be close to 0 (up to some error), while if the task is not identified the corresponding value would not be 0; the rest of the rows would be 1. We have created one vector for each task, which has 1 in the position of the corresponding task if the task was identified in the context.

Averaging and task superposition. As a last step, we average all the task identifiers and place the result in the last column, in which the next prediction will happen. We then use the averaged task identifier to weight the prediction of each task based on it, as in task superposition. If the task has been identified multiple times in the context, it would be assigned a higher weight/probability.

6 TASK SUPERPOSITION THROUGH THE LENS OF TASK VECTORS

While in Section 5 we provide an existential result by constructing a Transformer that performs task superposition and shows that task superposition is well within the expressive power of Transformers, we would like to further investigate how task superposition manifest in pretrained LLMs internally. In this section we explore the underlying mechanisms that LLMs employ during task superposition. In particular, we focus our empirical study on *task vectors* (Hendel et al., 2023) where the detailed implementation is in Appendix C. Task vectors are vectors in the embedding space and are found to encode the algorithm that a model internally implements to solve a task given in-context demonstrations.

²This step is not mandatory, but it ensures that we have no values over which we have no control. We leave as future work an error analysis on how these values could affect the task identifiers.

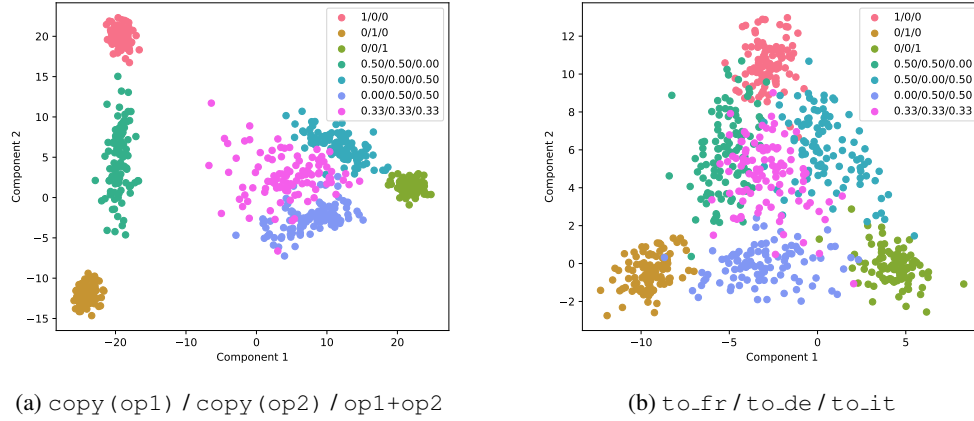


Figure 4: Task vectors of Llama-3 8B projected onto two axes chosen by LDA for two sets of tasks: **(a)** `copy (op1)`, `copy (op2)` and `op1+op2` and **(b)** `to_fr`, `to_de` and `to_it`. For tasks t_1, t_2, t_3 , we use “ $\mathbb{P}(t_1)/\mathbb{P}(t_2)/\mathbb{P}(t_3)$ ” to denote different levels of task mixtures, e.g., “0.50/0.50/0.00” represents the case where the in-context task examples are 50% t_1 , 50% t_2 and 0% t_3 .

We want to investigate if there is any relation between the task vectors of each individual task and the task vectors of a mixture of task examples in the prompt. To this end, we consider two sets of tasks:

- (a) `copy (op1)`, `copy (op2)` and `op1+op2` as in Figure 1b (left).
- (b) Given a two-digit integer, task `to_fr` translates it to French, task `to_de` translates it to German and task `to_it` translates it to Italian.

For each set of tasks, we collect the task vectors for each individual task and task vectors extracted from prompts that contain examples of different tasks. In Figure 4, we project task vectors along two axes chosen by linear discriminant analysis (LDA).

Finding 3: *LLMs internally combine task vectors during task superposition.*

Interestingly, we observe that the locations of task vectors of a mixture of tasks strongly correlate with the locations of task vectors for each individual task and the in-context task example distribution (the mixture ratio for examples of different tasks). For example, if the prompt includes an equal number of in-context examples from each task, the task vectors are roughly centered in the middle; if the prompt only contains in-context examples of two tasks, then the task vectors roughly lie on the connecting line between task vectors of two individual tasks. We argue that this observation is indicative of the fact that, when prompted with a mixture of in-context task examples, LLMs internally combine task vectors.

As we observe signs that LLMs internally compose task vectors, we want to further investigate whether we can reproduce the task superposition phenomenon by patching in a convex combination of task vectors. For example, for tasks `copy (op1)` and `copy (op2)`, we first extract the corresponding task vectors $V_{\text{copy (op1)}}$ and $V_{\text{copy (op2)}}$ on Llama-3 8B using the method described in Appendix C. We then make a convex combination of the two task vectors with parameter λ that controls the ratio:

$$V_{\text{interpolate}, \lambda} = \lambda \cdot V_{\text{copy (op1)}} + (1 - \lambda) \cdot V_{\text{copy (op2)}}.$$

Finding 4: *Convex combinations of task vectors produce task superposition.*

For a new query (in this scenario in the form “`{op1}@{op2}=`”), we patch the vector $V_{\text{interpolate}, \lambda}$ into the model at the task vector layer. We calculate the model output probabilities that correspond to each task while we vary λ . For each λ , we use 100 different queries and plot the average probabilities in the top row of Figure 5. As a comparison, in the bottom rows of Figure 5, we plot the corresponding output probabilities when providing the models with prompts containing mixture of task examples where the mixture ratio is controlled by λ .

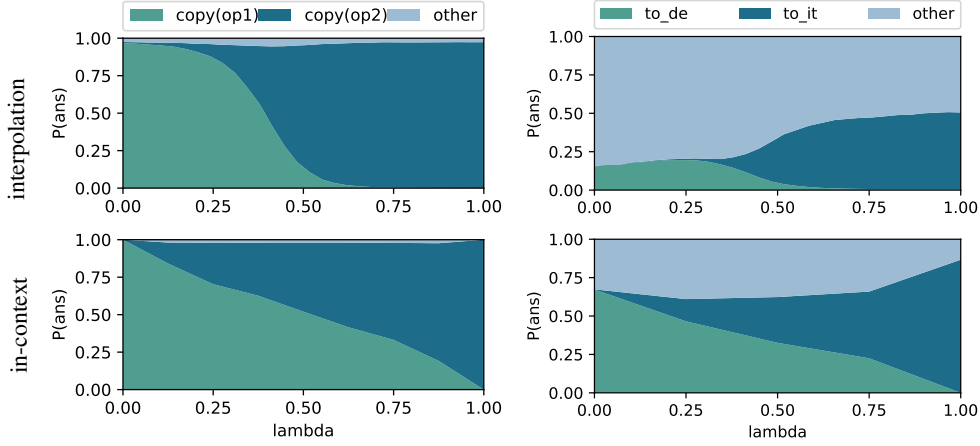
(a) Tasks: `copy (op1)` and `copy (op2)`(b) Tasks: translate `to_de` and `to_it`

Figure 5: On Llama-3 8B, we vary the proportion, λ , between two tasks and observe how the output probabilities for the correct answers change. The proportion λ is varied in two ways: (1) in the top row, we plot the output from patching in a convex combination of task vectors for two tasks. (2) in the bottom row, we plot the output from a mixed proportion of in-context examples for the two tasks. Subplot (a) shows the output probabilities from mixing two copy tasks and (b) shows the probabilities from mixing two translate tasks.

In top row of Figure 5, we observe that patching convex combinations of task vectors into the model produces task superposition. We would also like to point out that in Figure 5b, although irrelevant outputs sum up to a large probability, the task answers for two tasks `to_de` and `to_it` in most cases will still be the top-2 answers.

Comparing the top rows and the bottom rows, we can see that top rows (the scenario of interpolating task vectors of individual tasks) have larger probabilities of irrelevant output (category `other`). Task vector interpolation also produces less of a linear relationship between λ and the output probabilities. This shows that while convex combinations of task vectors are sufficient for producing task superposition, the convex combination does not fully explain task superposition. We leave it to future work to investigate other mechanistic explanations of task superposition.

7 TASK SUPERPOSITION CAPABILITIES AS THE MODEL SCALES

Finding 5: *Within the same LLM family, bigger models can solve more tasks in parallel and better calibrate to ICL distribution.*

We want to further investigate how models’ task superposition capabilities change as the model size scales. In particular, we investigate two questions: 1) whether larger models can perform more tasks in-context and 2) whether larger models can align their output distribution more closely with the distribution of task examples provided in the prompt. We chose the Qwen-1.5 model family since it contains several model sizes ranging from 0.5B to 14B parameters.

We first introduce a quantity which captures the capability of a model to perform multiple tasks. Given a prompt that contains examples of K tasks, we define r to be the number of these tasks whose correct answers appear among the model’s top- K most likely outputs. Note that $r \leq K$.

To see how close the model align the output distribution with the distribution of task examples, we use KL-divergence defined below:

$$\text{KL}(\mathcal{P}||\mathcal{D}) = \sum_{x \in X} \mathcal{P}(x) \log \left(\frac{\mathcal{P}(x)}{\mathcal{D}(x)} \right), \quad (2)$$

where $\mathcal{P}$ is the models’ probabilities on the outputs when correctly performing each task on the query and $\mathcal{D}$ is the in-context task example distribution. For example the prompt in Figure 1a (left) gives $\mathcal{P} = [0.5217, 0.1316, 0.1110, 0.2169, \dots]$ and $\mathcal{D} = [0.25, 0.25, 0.25, 0.25, 0, \dots]$.

We consider the setting of $K = 6$ different tasks: given an input of the form “{num}→” where num is a two-digit integer, we consider 6 tasks that output (1) num itself, (2) negation of num, (3) num + 1, (4) num − 1, (5) num × 2 and (6) num².

We choose the number of in-context examples $m = 60$ (each task has 10 examples) and configure the prompt with three different in-context task example distributions $\mathcal{D}_1, \mathcal{D}_2$ and $\mathcal{D}_3$. In particular, $\mathcal{D}_1$ is the uniform distribution, $\mathcal{D}_2$ has probability 0.5 on the third task and 0.1 on other tasks, and $\mathcal{D}_3$ is a distribution with probabilities alternating between 0.25 and 0.083.

For each in-context task examples distribution $\mathcal{D}_i$, we generate 100 prompts and for each prompt we calculate the probabilities of outputs when correctly performing each task. The average values of r and KL-divergence under three distributions are shown in Figure 6.

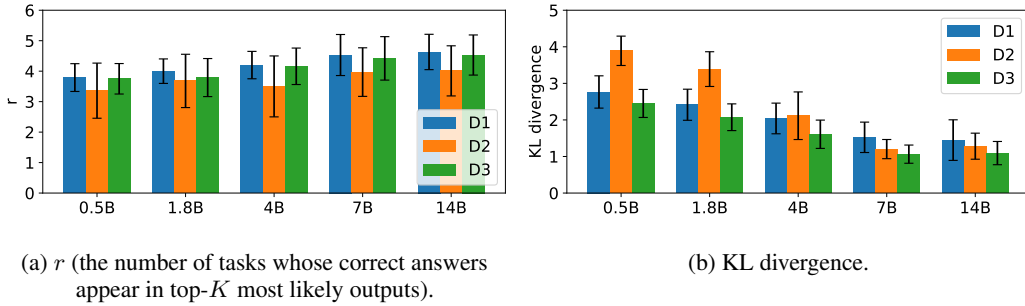


Figure 6: (a) Average number of tasks completed, r , and (b) KL divergence for Qwen-1.5 model family under ICL distributions $\mathcal{D}_1, \mathcal{D}_2$ and $\mathcal{D}_3$ where $\mathcal{D}_1$ is the uniform distribution, $\mathcal{D}_2$ has probability 0.5 on the third task and 0.1 on other tasks, and $\mathcal{D}_3$ is a distribution with probabilities alternating between 0.25 and 0.083.

In Figure 6a, we can observe that bigger models have higher r values (except for task distribution $\mathcal{D}_2$, 4B model has slightly lower r than that of the 1.8B model). This shows bigger models will have more correct answers of tasks show up in their top- K probable outputs and therefore they can solve more tasks at the same time. In Figure 6b, we can see that for larger models like Qwen-1.5 7B and Qwen-1.5 14B, the KL-divergence values are small, and for each model, the differences between KL-divergence values under in-context task example distributions $\mathcal{D}_1, \mathcal{D}_2$ and $\mathcal{D}_3$ are small. This indicates that bigger models can better calibrate their output distribution to the in-context task example distribution.

8 LIMITATIONS AND FUTURE DIRECTIONS

One limitation of our work is the current gap between the demonstrated capability of LLMs to perform task superposition and its practical application in real-world scenarios. While we have shown that LLMs possess the capacity to execute multiple tasks simultaneously, conventional decoding algorithms are not equipped to fully leverage this capability. This limitation stems from what we term “generation collapse,” a phenomenon where, after the first token is generated, the model tends to converge on predicting tokens for a single task, effectively negating its ability for multi-task execution.

This collapse presents a substantial challenge in harnessing the full power of task superposition. It highlights a critical area for future research: developing decoding strategies that can maintain the model’s multi-task state throughout the generation process. Recent work by Shen et al. (2024b) offers some hope that this direction may be fruitful, by proposing a “superposed decoding” algorithm. Their method efficiently generates multiple streams of tokens from a single inference pass by utilizing superposed token embeddings. While this approach represents a significant step forward, it also highlights the potential for further innovation in this area.

9 CONCLUSION

We report on the discovery of task superposition, which is the ability of LLMs to simultaneously solve distinct tasks from in-context examples. Task superposition is present in a variety of pretrained models, and becomes more accurate at predicting the distribution of tasks as the model size increases. We also find evidence that while displaying task superposition, models internally mix the task vectors of each individual task. We hope that our findings will contribute to understanding in-context learning mechanisms and enhance our knowledge of LLMs overall.

REFERENCES

- Rishabh Agarwal, Avi Singh, Lei M. Zhang, Bernd Bohnet, Stephanie Chan, Ankesh Anand, Zaheer Abbas, Azade Nova, John D. Co-Reyes, Eric Chu, Feryal Behbahani, Aleksandra Faust, and Hugo Larochelle. Many-shot in-context learning, 2024.
- Kwangjun Ahn, Xiang Cheng, Hadi Daneshmand, and Suvrit Sra. Transformers learn to implement preconditioned gradient descent for in-context learning. *arXiv preprint arXiv:2306.00297*, 2023.
- AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? Investigations with linear models, May 2023. URL <http://arxiv.org/abs/2211.15661>. arXiv:2211.15661 [cs].
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023a.
- Yu Bai, Fan Chen, Huan Wang, Caiming Xiong, and Song Mei. Transformers as statisticians: Provable in-context learning with in-context algorithm selection, 2023b.
- Yu Bai, Heyan Huang, Cesare Spinoso-Di Piano, Marc-Antoine Rondeau, Sanxing Chen, Yang Gao, and Jackie Chi Kit Cheung. Identifying and analyzing task-encoding tokens in large language models. (arXiv:2401.11323), February 2024. doi: 10.48550/arXiv.2401.11323. URL <http://arxiv.org/abs/2401.11323>. arXiv:2401.11323 [cs].
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html.
- Stephanie Chan, Adam Santoro, Andrew Lampinen, Jane Wang, Aaditya Singh, Pierre Richemond, James McClelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. *Advances in Neural Information Processing Systems*, 35:18878–18891, 2022.
- Brian Cheung, Alex Terekhov, Yubei Chen, Pulkit Agrawal, and Bruno Olshausen. Superposition of many models into one, June 2019. URL <http://arxiv.org/abs/1902.05522>. arXiv:1902.05522 [cs].

- Hyeong Kyu Choi and Yixuan Li. Picle: Eliciting diverse behaviors from large language models with persona in-context learning. (arXiv:2405.02501), May 2024. doi: 10.48550/arXiv.2405.02501. URL <http://arxiv.org/abs/2405.02501>. arXiv:2405.02501 [cs].
- Damai Dai, Yutao Sun, Li Dong, Yaru Hao, Zhifang Sui, and Furu Wei. Why can gpt learn in-context? language models secretly perform gradient descent as meta optimizers. *arXiv preprint arXiv:2212.10559*, 2022.
- Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Shashank Rajput, Michael Gira, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. Lift: Language-interfaced fine-tuning for non-language machine learning tasks. *arXiv preprint arXiv:2206.06565*, 2022.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy Models of Superposition, September 2022. URL <http://arxiv.org/abs/2209.10652>. arXiv:2209.10652 [cs].
- Fabian Falck, Ziyu Wang, and Chris Holmes. Is in-context learning in large language models bayesian? a martingale perspective. (arXiv:2406.00793), June 2024. URL <http://arxiv.org/abs/2406.00793>. arXiv:2406.00793 [cs, stat].
- Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers, 2023.
- Riccardo Grazi, Julien Siems, Simon Schrod, Thomas Brox, and Frank Hutter. Is mamba capable of in-context learning? (arXiv:2402.03170), April 2024. doi: 10.48550/arXiv.2402.03170. URL <http://arxiv.org/abs/2402.03170>. arXiv:2402.03170 [cs].
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. (arXiv:2312.00752), December 2023. doi: 10.48550/arXiv.2312.00752. URL <http://arxiv.org/abs/2312.00752>. arXiv:2312.00752 [cs].
- Roe Hendel, Mor Geva, and Amir Globerson. In-Context Learning Creates Task Vectors, October 2023. URL <http://arxiv.org/abs/2310.15916>. arXiv:2310.15916 [cs].
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing Models with Task Arithmetic, March 2023. URL <http://arxiv.org/abs/2212.04089>. arXiv:2212.04089 [cs].
- Janus. Simulators, 2022. URL <https://www.lesswrong.com/posts/vJFdjigzmcXMhNTsx/>.
- Andrew K Lampinen, Ishita Dasgupta, Stephanie CY Chan, Kory Matthewson, Michael Henry Tessler, Antonia Creswell, James L McClelland, Jane X Wang, and Felix Hill. Can language models learn from explanations in context? *arXiv preprint arXiv:2204.02329*, 2022.
- Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, et al. In-context reinforcement learning with algorithm distillation. *arXiv preprint arXiv:2210.14215*, 2022.
- Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context llms struggle with long in-context learning. (arXiv:2404.02060), April 2024. doi: 10.48550/arXiv.2404.02060. URL <http://arxiv.org/abs/2404.02060>. arXiv:2404.02060 [cs].
- Ziqian Lin and Kangwook Lee. Dual Operating Modes of In-Context Learning, February 2024. URL <http://arxiv.org/abs/2402.18819>. arXiv:2402.18819 [cs].
- Sheng Liu, Haotian Ye, Lei Xing, and James Zou. In-context vectors: Making in context learning more effective and controllable through latent space steering. (arXiv:2311.06668), February 2024. doi: 10.48550/arXiv.2311.06668. URL <http://arxiv.org/abs/2311.06668>. arXiv:2311.06668 [cs].

- Xinxi Lyu, Sewon Min, Iz Beltagy, Luke Zettlemoyer, and Hannaneh Hajishirzi. Z-icl: Zero-shot in-context learning with pseudo-demonstrations. *arXiv preprint arXiv:2212.09865*, 2022.
- Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? (arXiv:2202.12837), October 2022. doi: 10.48550/arXiv.2202.12837. URL <http://arxiv.org/abs/2202.12837>. arXiv:2202.12837 [cs].
- moire. Language models are multiverse generators, January 2021. URL <https://generative.ink/posts/language-models-are-multiverse-generators/>.
- Vishvak Murahari, Carlos E. Jimenez, Runzhe Yang, and Karthik Narasimhan. Datamux: Data multiplexing for neural networks. (arXiv:2202.09318), November 2022. doi: 10.48550/arXiv.2202.09318. URL <http://arxiv.org/abs/2202.09318>. arXiv:2202.09318 [cs].
- Cleo Nardo. The waluigi effect (mega-post), 2023. URL <https://www.lesswrong.com/posts/D7PumeYTDPfBTp3i7/the-waluigi-effect-mega-post>.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- Johannes Von Oswald, Eyvind Niklasson, Ettore Randazzo, Joao Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers Learn In-Context by Gradient Descent. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 35151–35174. PMLR, July 2023. URL <https://proceedings.mlr.press/v202/von-oswald23a.html>. ISSN: 2640-3498.
- Madhur Panwar, Kabir Ahuja, and Navin Goyal. In-context learning through the bayesian prism, June 2023. URL <https://arxiv.org/abs/2306.04891v2>.
- Jongho Park, Jaeseung Park, Zheyang Xiong, Nayoung Lee, Jaewoong Cho, Samet Oymak, Kangwook Lee, and Dimitris Papailiopoulos. Can mamba learn how to learn? a comparative study on in-context learning tasks. (arXiv:2402.04248), April 2024. doi: 10.48550/arXiv.2402.04248. URL <http://arxiv.org/abs/2402.04248>. arXiv:2402.04248 [cs].
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Allan Raventos, Mansheej Paul, Feng Chen, and Surya Ganguli. The effects of pretraining task diversity on in-context learning of ridge regression. In *ICLR Workshop on Mathematical and Empirical Understanding of Foundation Models (ME-FoMo)*, 2023.
- Gautam Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=aN4Jf6Cx69>.
- Laria Reynolds and Kyle McDonell. Multiversal views on language models, February 2021. URL <http://arxiv.org/abs/2102.06391>. arXiv:2102.06391 [cs].
- Murray Shanahan, Kyle McDonell, and Laria Reynolds. Role play with large language models. *Nature*, 623(7987):493–498, 2023.
- Ethan Shen, Alan Fan, Sarah M. Pratt, Jae Sung Park, Matthew Wallingford, Sham M. Kakade, Ari Holtzman, Ranjay Krishna, Ali Farhadi, and Aditya Kusupati. Superposed decoding: Multiple generations from a single autoregressive inference pass. (arXiv:2405.18400), May 2024a. doi: 10.48550/arXiv.2405.18400. URL <http://arxiv.org/abs/2405.18400>. arXiv:2405.18400 [cs].
- Ethan Shen, Alan Fan, Sarah M Pratt, Jae Sung Park, Matthew Wallingford, Sham M Kakade, Ari Holtzman, Ranjay Krishna, Ali Farhadi, and Aditya Kusupati. Superposed decoding: Multiple generations from a single autoregressive inference pass. *arXiv preprint arXiv:2405.18400*, 2024b.

- Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. (arXiv:2310.15213), February 2024. doi: 10.48550/arXiv.2310.15213. URL <http://arxiv.org/abs/2310.15213>. arXiv:2310.15213 [cs].
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Colin Wei, Yining Chen, and Tengyu Ma. Statistically meaningful approximation: a case study on approximating turing machines with transformers. *Advances in Neural Information Processing Systems*, 35:12071–12083, 2022.
- Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, et al. Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846*, 2023.
- Jingfeng Wu, Difan Zou, Zixiang Chen, Vladimir Braverman, Quanquan Gu, and Peter L Bartlett. How many pretraining tasks are needed for in-context learning of linear regression? In *International Conference on Learning Representations (ICLR)*, 2024.
- Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. (arXiv:2111.02080), July 2022. doi: 10.48550/arXiv.2111.02080. URL <http://arxiv.org/abs/2111.02080>. arXiv:2111.02080 [cs].
- Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. *arXiv preprint arXiv:2311.12424*, 2023.
- Yuchen Zeng, Wonjun Kang, Yicong Chen, Hyung Il Koo, and Kangwook Lee. Can mllms perform text-to-image in-context learning? (arXiv:2402.01293), April 2024. doi: 10.48550/arXiv.2402.01293. URL <http://arxiv.org/abs/2402.01293>. arXiv:2402.01293 [cs].
- Yufeng Zhang, Fengzhuo Zhang, Zhuoran Yang, and Zhaoran Wang. What and how does in-context learning learn? bayesian model averaging, parameterization, and generalization. (arXiv:2305.19420), October 2023. doi: 10.48550/arXiv.2305.19420. URL <http://arxiv.org/abs/2305.19420>. arXiv:2305.19420 [cs, stat].
- Hattie Zhou, Azade Nova, Hugo Larochelle, Aaron Courville, Behnam Neyshabur, and Hanie Sedghi. Teaching algorithmic reasoning via in-context learning. *arXiv preprint arXiv:2211.09066*, 2022.
- Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? a study in length generalization. *arXiv preprint arXiv:2310.16028*, 2023.

A NOTATIONS

Notation	Description
K	Number of tasks
l	Length of a task's output
ℓ	layer ℓ for a model
m	Number of in-context examples
n	Length of each in-context example
$\mathcal{V}$	Token vocabulary
$g_i(\cdot)$	Operation performed by Task i
$\mathbf{x}^{(j)}$	Data for example j
$\mathbf{y}^{(j)}$	Label for example j
$\mathbf{s}_m$	m in-context examples
$\mathbf{f}(\cdot)$	Model (predictor)
$\mathbf{p}$	Positional encodings

B IMPLEMENTATION DETAILS ON CALCULATING PROBABILITIES

In this section we provide details on how we calculate probabilities of different outputs given a prompt in our setting.

Notations. Let $\mathcal{V}$ be the token vocabulary, M be an LLM, T be the tokenizer. We use “...” to represent a string, $\langle \dots \rangle$ to represent a single token where the content within the angle brackets is an integer representing token’s index in vocabulary. For example, token $\langle 266 \rangle$ corresponds to “at”. We use $[\langle \dots \rangle, \dots, \langle \dots \rangle]$ to represent a sequence of tokens. Given a tokenizer, we use two functions $\text{tok}(\cdot)$ and $\text{detok}(\cdot)$ to tokenize strings and detokenize tokens. For example $\text{tok}(\text{“superposition”}) = [\langle 9712 \rangle, \langle 3571 \rangle]$ and $\text{detok}([\langle 16 \rangle, \langle 10 \rangle, \langle 16 \rangle, \langle 28 \rangle, \langle 17 \rangle]) = \text{“1+1=2”}$.

In our in-context learning setting, an input string consists of in-context examples (separated by the delimiter “\n”) and a query. For example, an example prompt can be “1+1=2\n2+2=4\n3+3=”.

We view an LLM as a next-token predictor and there is a corresponding $\mathbb{P}(\cdot|\cdot)$ such that given a sequence of tokens $[v_1, \dots, v_M]$ where $v_j \in \mathcal{V}$, $\mathbb{P}(u | [v_1, \dots, v_M])$ measures the probability of the next token being u where $u \in \mathcal{V}$.

Measuring the probabilities of task answers. Let I be the input prompt. For example, in the example in Figure 1a (left), the prompt is “11+26->37\n33+13->quarante-six\n...30+25->fifty-five\n91+83->”. We consider four tasks: 1) numerical addition, 2) addition in English, 3) addition in French and 4) addition in Spanish. The corresponding task answers (the output of correctly performing task on the query) are “174”, “one hundred and seventy-four”, “cent soixante-quatorze” and “ciento setenta y cuatro”, respectively. We want to measure the probability of each task answer.

Let o be a task answer in string. Let $[v_1, \dots, v_M] := \text{tok}(I)$ and let $[u_1, \dots, u_N] := \text{tok}(o)$. Then the probability of the task answer o given prompt I can be calculated as

$$\mathbb{P}(u_1 | [v_1, \dots, v_M]) \prod_{j=2}^N \mathbb{P}(u_j | [v_1, \dots, v_M, u_1, \dots, u_{j-1}]). \quad (3)$$

C IMPLEMENTATION DETAILS ON TASK VECTORS

We use the task vector definition from Hendel et al. (2023). For example, for task `copy (op1)` in Figure 1b (left), the procedure to collect the task vector consists of

1. Collect a dataset of 100 ICL sample prompts. Each prompt consists of $m = 60$ in-context examples of a particular task and a query $\mathbf{x}^{(m+1)}$. Each task example $(\mathbf{x}^{(j)}, \mathbf{y}^{(j)})$ follows the form “ $\{\text{op1}\} @ \{\text{op2}\} = \{\text{op1}\}$ ”, where $\mathbf{x}^{(j)}$ has the form “ $\{\text{op1}\} @ \{\text{op2}\} =$ ” and $\mathbf{y}^{(j)}$ is performing task `copy (op1)` on $\mathbf{x}^{(j)}$, namely `op1`.
2. For each prompt $\equiv \mathbf{s} = [\mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(m)}, \mathbf{y}^{(m)}, \mathbf{x}^{(m+1)}]$ in the dataset, we feed $\mathbf{s}$ into the transformer model $\mathbf{f}$, and extract the feature (which is a vector) at the last “=” token in layer ℓ . Call this vector $\mathbf{f}(\mathbf{s}; \ell)$. Then we average $\mathbf{f}(\mathbf{s}; \ell)$ across all prompt $\mathbf{s}$ to get $\mathbf{v}(\ell)$ for layer ℓ .
3. Now for each layer ℓ we have a vector $\mathbf{v}(\ell)$. We run a forward pass with one query $\mathbf{x}$ in the form “ $\{\text{op1}\} @ \{\text{op2}\} =$ ” and we patch in $\mathbf{v}(\ell)$ at the “=” token position in layer ℓ , simulating the effect of a complete context. We repeat this process 100 times for different query $\mathbf{x}$ and get an accuracy acc_ℓ of performing task `copy (op1)` with vector $\mathbf{v}(\ell)$.
4. The task vector layer ℓ^* is selected by

$$\ell^* = \arg \max_{\ell} \text{acc}_\ell,$$

and we define the task vector $\mathbf{V}_{\text{copy (op1)}} := \mathbf{v} = \mathbf{v}(\ell^*)$.

Here we record the task vector layer where task vectors are extracted in Section 6.

Task	Task vector layer
<code>copy (op1)</code> , <code>copy (op2)</code> , <code>op1+op2</code>	14
<code>to_de (op1)</code> , <code>to_fr (op1)</code> , <code>to_it (op1)</code>	19

Table 1: Task vector layer for various tasks considered in Section 6.

D CONSTRUCTION DISPLAYING SUPERPOSITION

In this section we construct a Transformer that is performing superposition of multiple tasks at inference. For this purpose, we first construct a Transformer that copies from n -tuple in-context examples the i -th one, as well as any function using the ReLU layers. We then create indicator vectors, for each task, which show whether a specific task is present in-context or not. As a last step, we combine these indicator vectors to create the superposition of different tasks. Notice that using the parallel heads of the transformer architecture we can process each task independently until the last step in which the predictions are combined.

D.1 OVERVIEW

Here we provide a brief overview of how the construction is implemented, while latter we provide the corresponding details.

Prediction based on multiple tasks. Assume that we are given m in-context samples $(\mathbf{x}_1^{(j)}, \dots, \mathbf{x}_{n-2}^{(j)}, '=', \mathbf{y}^{(j)})_{j=1}^m$ where '=' represents a specific value used only for preceding the label, and a set of k different Transformers TF_i which can implement the T different desired tasks, where each deterministic task is denoted as $\mathbf{g}_i(\mathbf{x}^{(j)})$ with $i \in [k]$ and $j \in [m]$, *i.e.* $\mathbf{y}^{(j)} = \mathbf{g}_i(\mathbf{x}^{(j)})$ for some task i dependent on sample j . Using the weights of each TF_i , we can compute outputs of the following form:

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \end{bmatrix} \rightarrow \begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \|\mathbf{g}_1(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \|\mathbf{g}_T(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \end{bmatrix}$$

We use the l_1 norm to aggregate the prediction, in case that the task is multi-dimensional. These differences are used to identify tasks, as $\|\mathbf{g}_i(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 \approx 0$ for $\mathbf{y}^{(j)}$ coming from task i . Different heads at each layer in the model are used to execute each of the tasks in parallel using the weights from TF_i . In Appendix D we construct tasks where an arbitrary function $\mathbf{g}_i(\mathbf{x}_l^{(j)})$ is implemented using ReLUs for some fixed l that is task-specific.

Creating task identifiers. Having the differences between the implemented function and the label, we first use the ReLUs to clean up the vectors $\mathbf{v}_k$ so that only the positions in each vector that are associated with a task are maintained and the rest are set to 1³. We thus create the vectors $(\mathbf{v}'_k)_i = \|\mathbf{g}_k(\mathbf{x}_{1:l-1}^{(j)}) - \mathbf{x}_l^{(j)}\|_1$ and $(\mathbf{v}'_*)_* = 1$ otherwise. Now we use ReLUs to threshold and create an indicator vectors $\mathbf{1}_{\{\|\mathbf{g}_k(\mathbf{x}_{1:l-1}^{(j)}) - \mathbf{x}_l^{(j)}\|_1 \approx 0\}}$ which identify the task, *i.e.* these are task identifiers.

Notice that if the task is correctly predicted then the difference should be close to 0 (up to some error), while if the task is not identified the corresponding value would not be 0; the rest of the rows would be 1. We have created one vector for each task, which has 1 in the position of the corresponding task if the task was identified in the context.

Averaging and task superposition. As a last step, we average all the task identifiers and place the result in the last column, in which the next prediction will happen. We then use the averaged task identifier to weight the prediction of each task based on it, as in task superposition. If the task has been identified multiple times in the context, it would be assigned a higher weight/probability.

D.2 TASK IDENTIFICATION

The first task for performing task superposition based on in-context examples is to define a set of tasks that the model is able to implement.

First, the outputs of tasks need to be identified.

³This step is not mandatory, but it ensures that we have no values over which we have no control. We leave as future work an error analysis on how these values could affect the task identifiers

Lemma 1. Consider the following input

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{y}^{(j-1)} & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 1 & 0 & 0 & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \end{bmatrix},$$

where $\mathbf{x}_i^{(j)} \in \mathbb{R}^{d-1}$ before the positional encodings are added, with one additional dimension that represents if the symbol is an ‘equals’ symbol. Then, a 1-layer transformer with a single attention head and embedding dimension $\mathcal{O}(d + \log(mn))$ can output

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{y}^{(j-1)} & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 1 & 0 & \dots & 0 & 0 & 1 & 0 & \dots \end{bmatrix}$$

Proof. With positional encodings appended, let the input have the following structure:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 1 & 0 & 0 & \dots \\ \mathbf{p}_{n+1} & \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \mathbf{p}_{(j+1)n+1} & \dots \\ \mathbf{p}_n & \dots & \mathbf{p}_{jn} & \mathbf{p}_{jn+1} & \dots & \mathbf{p}_{jn+n-3} & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{(j+1)n} & \dots \end{bmatrix} \quad (4)$$

To rotate the second row one position to the right, use the following matrices.

$$\begin{aligned} \mathbf{W}_Q &= \begin{bmatrix} 0 & 0 & 0 & \mathbb{I} \end{bmatrix} \\ \mathbf{W}_K &= \begin{bmatrix} 0 & 0 & C\mathbb{I} & 0 \end{bmatrix} \\ \mathbf{W}_V &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

The pair $\mathbf{W}_Q$ and $\mathbf{W}_K$ attend tokens to the token directly to the right. The value matrix simply filters only the second row in-place. A second head can be used to clear the original 1s, resulting in

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 1 & 0 & \dots \\ \mathbf{p}_{n+1} & \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \mathbf{p}_{(j+1)n+1} & \dots \\ \mathbf{p}_n & \dots & \mathbf{p}_{jn} & \mathbf{p}_{jn+1} & \dots & \mathbf{p}_{jn+n-3} & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{(j+1)n} & \dots \end{bmatrix}, \quad (5)$$

as desired. $\square$

Implementation of functions. To illustrate a set of operations that could be implemented with a transformer, we consider approximating functions as sums of ReLUs; we use a result from Bai et al. (2023b), which we present below.

Definition 1 (Definition 12 in Bai et al. (2023b)). A function $g : \mathbb{R}^k \rightarrow \mathbb{R}$ is (ϵ, R, M, C) -approximable by sum of ReLUs, if there exists an “ (M, C) -sum of ReLUs” function

$$f_{M,C}(\mathbf{x}) = \sum_{m=1}^M c_m \text{ReLU}(\mathbf{a}_m^\top [\mathbf{x}; 1]) \text{ with } \sum_{m=1}^M |c_m| \leq C, \max_{m \in [M]} \|\mathbf{a}_m\|_1 \leq 1, \mathbf{a}_m \in \mathbb{R}^{k+1}, c_m \in \mathbb{R}$$

such that $\sup_{\mathbf{x} \in [-R, R]^k} |g(\mathbf{x}) - f_{M,C}(\mathbf{x})| \leq \epsilon$.

Definition 2 (Definition A.1 in Bai et al. (2023b)). We say a function $g : \mathbb{R}^k \rightarrow \mathbb{R}$ is (R, C_l) -smooth if for $s = \lceil (k-1)/2 \rceil + 2$, $g \in C^{24}$ on $[-R, R]^k$ and

$$\sup_{\mathbf{x} \in [-R, R]^k} \|\nabla^i g(\mathbf{x})\|_\infty = \sup_{\mathbf{x} \in [-R, R]^k} \max_{j_1, \dots, j_i \in [k]} |\partial_{x_{j_1}, \dots, x_{j_i}} g(\mathbf{x})| \leq L_i$$

for all $i = 0, 1, 2$ with $\max_{0 \leq i \leq s} L_i R^i \leq C_l$.

⁴ C^i denotes that a function is i times differentiable with continuous i -th derivative.

Proposition 1 (Proposition A.1 in Bai et al. (2023b)). *For any $\epsilon > 0$, $R \geq 1$, $C_l > 0$, we have that: Any (R, C_l) -smooth function, $g : \mathbb{R} \rightarrow \mathbb{R}$ is (ϵ, R, M, C') -approximable by sum of ReLUs (Definition 1) with $M \leq C(k)C_l^2 \log(1 + C_l\epsilon)/\epsilon^2$.*

Lemma 2. *For any function $g : \mathbb{R}^k \rightarrow \mathbb{R}$ that is (R, C_l) -smooth, there exists a transformer with two layers, one head and width $\mathcal{O}(\log(n) + d)$, where d satisfies the requirements of Prop. 1, such that given as input*

$$X = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{y}^{(j-1)} & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 1 & 0 & \dots & 0 & 0 & 1 & 0 & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \end{bmatrix},$$

it outputs

$$X = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{y}^{(j-1)} & \mathbf{x}_1^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ * & \dots & * & * & \dots & * & * & * & * & \dots \\ 0 & \dots & \tilde{g}(\mathbf{x}_i^{(j-1)}) - \mathbf{y}^{(j-1)} & * & \dots & * & * & \tilde{g}(\mathbf{x}_i^{(j)}) - \mathbf{y}^{(j)} & * & \dots \end{bmatrix}$$

where $|\tilde{g}(x) - g(x)| \leq \epsilon$ and for some $i \in [1, \dots, n-2]$.

Proof. We consider that the positional encodings are added in the input and we have

$$X = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 1 & 0 & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \\ 1 & \dots & 1 & 1 & \dots & 1 & 1 & 1 & 1 & \dots \\ \mathbf{p}_{jn+1} & \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \mathbf{p}_{(j+1)n+1} & \dots \\ \mathbf{p}_{n+1-s} & \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-2-s} & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \mathbf{p}_{(j+1)n+1-s} & \dots \end{bmatrix} \quad (6)$$

where we fix some positional encodings $\mathbf{p}_k$ where $\mathbf{p}_k^\top \mathbf{p}_k$ is larger than $\mathbf{p}_k^\top \mathbf{p}_l$ by some threshold for $k \neq l$. The encodings used here are the binary representations of $k \in \{-1, 1\}^{\log(mn)}$. Further, we consider 1s in the positions with the results of the task to differentiate the context of the task and the result of the task. Define $s = n - i$, the distance between the result and the associated value in the context.

In the first layer, we use the MLP's to create $\tilde{g}$ according to Proposition 1

$$X = \begin{bmatrix} \mathbf{x}_1^{(1)} & \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-2}^{(j)} & = & \mathbf{y}^{(j)} & \mathbf{x}_1^{(j+1)} & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 1 & 0 & \dots \\ \tilde{g}(\mathbf{x}_1^{(1)}) & \dots & \tilde{g}(\mathbf{x}_1^{(j)}) & \tilde{g}(\mathbf{x}_2^{(j)}) & \dots & \tilde{g}(\mathbf{x}_{n-2}^{(j)}) & \tilde{g}(=) & \tilde{g}(\mathbf{y}^{(j)}) & \tilde{g}(\mathbf{x}_1^{(j+1)}) & \dots \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \\ 1 & \dots & 1 & 1 & \dots & 1 & 1 & 1 & 1 & \dots \\ \mathbf{p}_{n+1} & \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \mathbf{p}_{(j+1)n+1} & \dots \\ \mathbf{p}_{n+1-s} & \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-2-s} & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \mathbf{p}_{(j+1)n+1-s} & \dots \end{bmatrix} \quad (7)$$

The next operation is a shift of the sequence of $\tilde{g}(\cdot)$'s to the right by s . This will align the desired output $\tilde{g}(\mathbf{x}_i^{(j)})$ with the observed output $\mathbf{y}^{(j)}$. Consider the following weight matrices

$$W_Q = [\dots \quad 0 \quad 0 \quad \mathbb{I}] \quad (8)$$

$$W_K = [\dots \quad 0 \quad C\mathbb{I} \quad 0] \quad (9)$$

$$W_V = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & \mathbb{I} & \dots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix} \quad (10)$$

for some large constant C to decrease error from the softmax attending to the incorrect tokens. This produces (within a small error induced by using a softmax)

$$(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X})_{i,j} = \mathbf{p}_{n+i}^\top \mathbf{p}_{n-s+j} \quad (11)$$

$$\sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X})_{i,j} = \mathbb{1}_{\{n+i=n-s+j\}} = \mathbb{1}_{\{i=j-s\}} \quad (12)$$

$$\mathbf{W}_V \mathbf{X} = \begin{bmatrix} \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots \\ \dots & \tilde{g}(\mathbf{x}_1^{(j)}) & \tilde{g}(\mathbf{x}_2^{(j)}) & \dots & \tilde{g}(\mathbf{x}_{n-2}^{(j)}) & \tilde{g}(=) & \tilde{g}(\mathbf{y}^{(j)}) & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots \end{bmatrix} \quad (13)$$

$$\mathbf{W}_V \mathbf{X} \sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X}) = \begin{bmatrix} \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots \\ \dots & * & * & \dots & * & * & \tilde{g}(\mathbf{x}_i^{(j)}) & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \end{bmatrix} \quad (14)$$

$$\mathbf{X} + \mathbf{W}_V \mathbf{X} \sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X}) = \begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-1}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 0 & 1 & \dots \\ \dots & * & * & \dots & * & * & \tilde{g}(\mathbf{x}_i^{(j)}) & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-2-s} & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (15)$$

$$(16)$$

Each matrix above only shows the slice that contains the j -th in-context example. This is repeated for each of the other in-context examples.

As a final step with an MLP, subtract row 1 from row 3 to achieve the following output:

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & \mathbf{x}_{n-1}^{(j)} & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 0 & 1 & \dots \\ \dots & * & * & \dots & * & * & \tilde{g}(\mathbf{x}_i^{(j)}) - \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-2} & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-2-s} & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (17)$$

□

Copy Tasks As has been experimentally investigated, the situation where a specific position within the context is copied as the label can be easily implemented by setting $g(\mathbf{x}) = \mathbf{x}$. The dependence on the subscript i within the construction is what allows the position copied to vary.

D.2.1 IDENTIFYING IF TASK'S OUTPUT MATCHES THE IN-CONTEXT EXAMPLE

Lemma 3. *A three layer transformer with ReLU MLPs and embedding dimension $\mathcal{O}(d + \log(mn))$ can calculate the proportion of in context examples that come from a specific task, where m is the number of in-context examples, each of length n and dimension d .*

Proof. We now have a matrix of the following form.

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 1 & \dots \\ \dots & * & * & \dots & * & f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (18)$$

If the task is correct, then $f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} \approx \mathbf{0}$, with some small error coming from softmaxs and function approximation error. First, we find the $L1$ -norm of $f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}$ using an MLP. For calculating $\|\mathbf{z}\|_1$ for arbitrary $\mathbf{z}$, we can use

$$\|\mathbf{z}\|_1 = \sum_{i=1}^d \text{ReLU}(z_i) - \text{ReLU}(-z_i) \quad (19)$$

which can be done in a single 1-layer MLP. Thus, we have

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 1 & \dots \\ \dots & * & * & \dots & * & f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} & \dots \\ \dots & * & * & \dots & * & \|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (20)$$

Notice that if some task has different dimension than another task, the “extra” rows would be zero and will not affect the result.

For clarity, we set all $*$ values in the $\|\cdot\|_1$ row to 1s. These will cause the following $\hat{\delta}$ in the following set these to 0. This operation can be omitted as the construction handles these trash values at a later layer.

Let b represent the value of the flag in the second row marking the $\mathbf{y}$ vectors and let x represent the values in the row with $\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1$. The following ReLUs set the $*$ values to 1.

$$x \leftarrow x + 1 - \text{ReLU}(x - Cb) - \text{ReLU}(Cb - C + 1) \quad (21)$$

for some large constant C . When $b = 0$, this reduces to $x + 1 - x = 1$, and when $b = 1$, this reduces to $x + 1 - 1 = x$, as desired.

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 1 & \dots \\ \dots & * & * & \dots & * & f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} & \dots \\ \dots & 1 & 1 & \dots & 1 & \|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (22)$$

Now define a thresholding function $\hat{\delta}(z)$ that satisfies $\hat{\delta}(0) = 1$ and $\hat{\delta}(z) = 0$ for $z \gg 0$. One such function used here is

$$\hat{\delta}_C(z) = \text{ReLU}(1 - Cz) \quad (23)$$

for some constant C , where larger C captures a narrower neighborhood of 0.

However, a slight change needs to be added to $\hat{\delta}_C$. In the same row as $\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|$ are many values that need to be discarded. Let b be the bit for the current column marking if the column contains an $\mathbf{x}$ or a $\mathbf{y}$. We use instead

$$\hat{\delta}_C(b, z) = \text{ReLU}(b - Cz) \quad (24)$$

This will be zero whenever $b = 0$ and $z \geq 0$. We then have as output

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \dots & = & \mathbf{y}^{(j)} & \dots \\ \dots & 0 & 0 & \dots & 0 & 1 & \dots \\ \dots & * & * & \dots & * & f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} & \dots \\ \dots & 1 & 1 & \dots & 1 & \|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \dots \\ \dots & 0 & 0 & \dots & 0 & \hat{\delta}_C(\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1) & \dots \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots \\ \dots & 1 & 1 & \dots & 1 & 1 & \dots \\ \dots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \dots & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \dots \\ \dots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \dots & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \dots \end{bmatrix} \quad (25)$$

Importantly, $\hat{\delta}_C(\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1) = 1$ when $f(\cdot)$ is the correct task and $\hat{\delta}_C(\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1) = 0$ when $f(\cdot)$ disagrees by more than $\frac{1}{C}$ in $L1$ -norm.

Lastly, for the next step in the construction, we need to average these soft indicators $\hat{\delta}$ to see how common f is within the context. This is done with an attention layer. Let $\mathbf{W}_Q$ select the row with all 1s multiplied by some large constant C , and let $\mathbf{W}_K$ select the row with flags for results $\mathbf{y}$. Then

$$\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X} = \begin{bmatrix} \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ C & C & \dots & C & C \\ \vdots & \vdots & & \vdots & \vdots \end{bmatrix} \quad (26)$$

$$\sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{X}) \approx \begin{bmatrix} \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ 1/m & 1/m & \dots & 1/m & 1/m \\ \vdots & \vdots & & \vdots & \vdots \end{bmatrix} \quad (27)$$

where a $1/m$ will appear in every row corresponding to a result $\mathbf{y}$. Let the value matrix select the row containing $\hat{\delta}(\|f(\mathbf{x}^{(j)} - \mathbf{y}^{(j)})\|_1)$. Denote $p = \frac{1}{m} \sum_{j=1}^m \hat{\delta}(\|f(\mathbf{x}^{(j)} - \mathbf{y}^{(j)})\|_1)$. Without causal masking, we would have as output

$$\begin{bmatrix}
\cdots & \mathbf{x}_1^{(j)} & \mathbf{x}_2^{(j)} & \cdots & = & \mathbf{y}^{(j)} & \cdots \\
\cdots & 0 & 0 & \cdots & 0 & 1 & \cdots \\
\cdots & * & * & \cdots & * & f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)} & \cdots \\
\cdots & 1 & 1 & \cdots & 1 & \|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1 & \cdots \\
\cdots & 0 & 0 & \cdots & 0 & \hat{\delta}_C(\|f(\mathbf{x}^{(j)}) - \mathbf{y}^{(j)}\|_1) & \cdots \\
\cdots & p & p & \cdots & p & p & \cdots \\
\cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} & \cdots \\
\cdots & 1 & 1 & \cdots & 1 & 1 & \cdots \\
\cdots & \mathbf{p}_{jn+1} & \mathbf{p}_{jn+2} & \cdots & \mathbf{p}_{jn+n-1} & \mathbf{p}_{jn+n} & \cdots \\
\cdots & \mathbf{p}_{jn+1-s} & \mathbf{p}_{jn+2-s} & \cdots & \mathbf{p}_{jn+n-1-s} & \mathbf{p}_{jn+n-s} & \cdots
\end{bmatrix} \quad (28)$$

However, with causal masking, we can only guarantee that p will appear in the columns containing the most recent example being queried. Thankfully, this is all that is needed. $\square$

D.3 TASK EXECUTION

Lemma 4. *A two layer transformer, with embedding dimension $\mathcal{O}(d + \log(mn))$ can perform a task and weight its output by the proportion of examples of that task seen within the context.*

Now that the proportions of each task have been identified in the context, the task itself needs to be executed for the new example being queried. To simplify notation, let the input to this step be

$$\mathbf{X} = \begin{bmatrix}
\cdots & \mathbf{x}_1^{(m)} & \mathbf{x}_2^{(m)} & \cdots & \mathbf{x}_{n-2}^{(m)} & = \\
\cdots & * & * & \cdots & * & * \\
\cdots & p & p & \cdots & p & p \\
\cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\
\cdots & * & * & \cdots & * & * \\
\cdots & 1 & 1 & \cdots & 1 & 1
\end{bmatrix} \quad (29)$$

Following the same process as outlined above, although with slightly different positional encodings, calculate $f(\mathbf{x}^{(m)})$ and place that result in the final column being decoded. These need to be added at the beginning of the construction, but are only introduced here for clarity.

$$\begin{bmatrix}
\cdots & \mathbf{x}_1^{(m)} & \mathbf{x}_2^{(m)} & \cdots & \mathbf{x}_{n-2}^{(m)} & \mathbf{x}_{n-1}^{(m)} \\
\cdots & * & * & \cdots & * & * \\
\cdots & p & p & \cdots & p & p \\
\cdots & * & * & \cdots & * & f(\mathbf{x}^{(m)}) \\
\cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\
\cdots & * & * & \cdots & * & * \\
\cdots & 1 & 1 & \cdots & 1 & 1 \\
\cdots & 0 & 0 & \cdots & 0 & 1 \\
\cdots & 0 & 0 & \cdots & 1 & 0
\end{bmatrix} \quad (30)$$

We will transform the row containing all p to be able to approximately multiply p by $f(\mathbf{x}^{(m)})$. Using the second to last row, perform $p \rightarrow 1 - p$. Using the last two rows, clear out the rest of that row and fill it with $-C$ for some large constant C . We then have

$$\begin{bmatrix}
\cdots & \mathbf{x}_1^{(m)} & \mathbf{x}_2^{(m)} & \cdots & \mathbf{x}_{n-2}^{(m)} & \mathbf{x}_{n-1}^{(m)} \\
\cdots & * & * & \cdots & * & * \\
\cdots & -C & -C & \cdots & p & 1 - p \\
\cdots & * & * & \cdots & * & f(\mathbf{x}^{(m)}) \\
\cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\
\cdots & * & * & \cdots & * & * \\
\cdots & 1 & 1 & \cdots & 1 & 1 \\
\cdots & 0 & 0 & \cdots & 0 & 1 \\
\cdots & 0 & 0 & \cdots & 1 & 0
\end{bmatrix} \quad (31)$$

Further, use the second-to-last row to clear out all $*$ in the rows below.

$$\begin{bmatrix} \dots & \mathbf{x}_1^{(m)} & \mathbf{x}_2^{(m)} & \dots & \mathbf{x}_{n-2}^{(m)} & \mathbf{x}_{n-1}^{(m)} \\ \dots & * & * & \dots & * & * \\ \dots & -C & -C & \dots & p & 1-p \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & f(\mathbf{x}^{(m)}) \\ \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} \\ \dots & * & * & \dots & * & * \\ \dots & 1 & 1 & \dots & 1 & 1 \\ \dots & 0 & 0 & \dots & 0 & 1 \\ \dots & 0 & 0 & \dots & 1 & 0 \end{bmatrix} \quad (32)$$

These previous operations can all be done in a single MLP.

Lastly, use an attention layer where $\mathbf{W}_K$ selects the row with the $-Cs$, $\mathbf{W}_Q$ selects the row with all 1s, and $\mathbf{W}_V$ selects the $f(\mathbf{x}^{(m)})$. For the last token $\mathbf{x}_L$,

$$\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{x}_L = \begin{bmatrix} \vdots \\ -C \\ -C \\ \vdots \\ p \\ 1-p \end{bmatrix} [1] = \begin{bmatrix} \vdots \\ -C \\ -C \\ \vdots \\ p \\ 1-p \end{bmatrix} \quad (33)$$

$$\sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{x}_L) \approx \begin{bmatrix} \vdots \\ -\infty \\ -\infty \\ \vdots \\ p \\ 1-p \end{bmatrix} = \begin{bmatrix} \vdots \\ 0 \\ 0 \\ \vdots \\ \frac{1}{1+e^{1-2p}} \\ 1 - \frac{1}{1+e^{1-2p}} \end{bmatrix} \quad (34)$$

$$\mathbf{W}_V \mathbf{x}_L \sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{x}_L) = \begin{bmatrix} \vdots \\ \mathbf{0} \\ \frac{1}{1+e^{1-2p}} \mathbf{0} + (1 - \frac{1}{1+e^{1-2p}}) f(\mathbf{x}^{(m)}) \\ \mathbf{0} \\ \vdots \end{bmatrix} \quad (35)$$

$$\mathbf{x}_L + \mathbf{W}_V \mathbf{x}_L \sigma_S(\mathbf{X}^\top \mathbf{W}_K^\top \mathbf{W}_Q \mathbf{x}_L) = \begin{bmatrix} \mathbf{x}_{n-1}^{(m)} \\ * \\ 1-p \\ \frac{1}{1+e^{1-2p}} f(\mathbf{x}^{(m)}) \\ \mathbf{0} \\ * \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad (36)$$

Importantly, we are left with $\frac{1}{1+e^{1-2p}} f(\mathbf{x}^{(m)})$. The factor $\frac{1}{1+e^{1-2p}}$ is approximately p , especially around $\frac{1}{2}$. This multiplication can also be calculated more accurately with approximations using ReLUs or sigmoids, but for brevity and following experimental evidence of a sigmoid shape in task superpositions, these options are omitted.

D.4 SUPERPOSED TASKS WITH PARALLEL HEADS

The above construction works for a single task, where the output is weighted by the proportions of the task within the context. To complete the construction of a transformer that does superposition of tasks, each of these models needs to be placed within the same overall transformer. This is described here.

Let there be a collection of tasks $\{t_i\}_{i=1}^T$ which can be executed by transformers with model weights represented by subscripts $(\cdot)_i$. With the input to each transformer being $\mathbf{X}^{(i)}$, the overall input matrix is given by vertically stacking these matrices.

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_{T-1} \\ \mathbf{X}_T \end{bmatrix} \quad (37)$$

Similarly, define each MLP's weights and biases as

$$\mathbf{W} = \text{diag}(\mathbf{W}_1, \dots, \mathbf{W}_T) \quad \mathbf{b} = \begin{bmatrix} \mathbf{b}_1 \\ \vdots \\ \mathbf{b}_T \end{bmatrix} \quad (38)$$

This puts every MLP to be independent of each other. Lastly, we need to change the attention layers. This requires the use of one head per task. In each of the following, $\mathbf{W}^{(i)}$ is a weight matrix for head i , $(\mathbf{W})_i$ is the weight matrix for task i in its individual transformer, and each matrix below is in the i -th block.

$$\mathbf{W}_V^{(i)} = \begin{bmatrix} \vdots \\ \mathbf{0} \\ (\mathbf{W}_V)_i \\ \mathbf{0} \\ \vdots \end{bmatrix}^\top \quad \mathbf{W}_K^{(i)} = \begin{bmatrix} \vdots \\ \mathbf{0} \\ (\mathbf{W}_K)_i \\ \mathbf{0} \\ \vdots \end{bmatrix}^\top \quad \mathbf{W}_Q^{(i)} = \begin{bmatrix} \vdots \\ \mathbf{0} \\ (\mathbf{W}_Q)_i \\ \mathbf{0} \\ \vdots \end{bmatrix}^\top \quad (39)$$

In all, this model executes multiple tasks in superposition by using parallel streams of heads that each performs a single task. Task identification can happen through the same mechanism as task execution by comparing the output of the task on each in context example with the true output.

For context related tasks, there needs to be positional encodings that allow for looking back a fixed number of tokens. For context agnostic tasks, a wide MLP can be used to approximate arbitrary non-linear transformations of the input. Each of these tasks only require a small number of layers, significantly smaller than those of modern LLMs. It may be possible that LLMs do certain tasks with different combinations of layers.

Also, if we take the feature p from each parallel stream, this creates the following task identifier.

$$\mathbf{v} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_T \end{bmatrix} \quad (40)$$

Interpolating between the pure tasks, represented by unit vectors, different amounts of each task will appear in the superposition in roughly equal proportions to those found in $\mathbf{v}$.

Lastly, we restate this construction formally.

Theorem 1. *A seven layer transformer with embedding dimension $\mathcal{O}(d + \log(mn))$ with K heads per attention layer can perform k tasks on vectors of dimension d in superposition, with weighting based on m different in-context examples each of length n .*

Proof. Using in succession each of Lemma 1, Lemma 2, Lemma 3, and Lemma 4, a transformer with the desired properties can execute k tasks in parallel. Lemma 1 identifies positions within the context that contain the labels $\mathbf{y}$. Lemma 2 then uses function approximation to perform arbitrary tasks within the architecture, which are then used by 3 to find the proportions of each task and aggregate them into a single task identifier. Lastly, Lemma 4 uses this task identifier to create a weighted sum of outputs from the different tasks based on their in-context proportions. $\square$

Remark. *Transformers of greater depth than seven layers can also represent this construction by setting the weights in all other layers for the non residual part to zero.*